

Chuyên đề: Bảng Thưa & Xử Lý Truy Vấn Tĩnh Đoạn $O(1)$ (Sparse Table & RMQ)

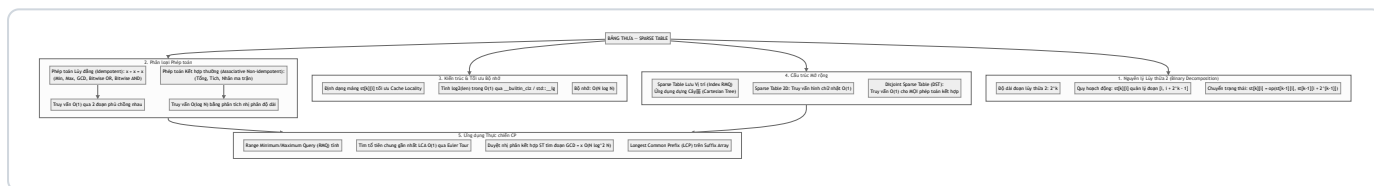
(Sparse Table, Idempotent Operations, Range Minimum Query, 2D Sparse Table & Disjoint Sparse Table)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy Toàn cảnh (Mindmap)



1. Đặt Vấn Đề & Bản Chất Thuật Toán

1.1. Bài toán Range Minimum Query (RMQ)

Cho một mảng tĩnh gồm N phần tử $A[0 \dots N - 1]$. Cần trả lời Q truy vấn có dạng:

$$\text{Query}(L, R) = \min_{L \leq i \leq R} A[i] \quad (0 \leq L \leq R < N)$$

Trong các bài thi lập trình thi đấu (ICPC, OLP), số lượng truy vấn Q thường rất lớn ($Q = 10^6 \dots 10^7$), trong khi mảng A không bị thay đổi (tĩnh - static).

Cấu trúc dữ liệu	Thời gian Tiền xử lý	Thời gian Truy vấn	Cập nhật điểm	Bộ nhớ
Duyệt trâu (Brute Force)	$O(1)$	$O(N)$	$O(1)$	$O(N)$
Bảng 2D đầy đủ (All Pairs)	$O(N^2)$	$O(1)$	Không hỗ trợ	$O(N^2)$
Cây Phân Đoạn (Segment Tree)	$O(N)$	$O(\log N)$	$O(\log N)$	$O(4N)$

Cấu trúc dữ liệu	Thời gian Tiền xử lý	Thời gian Truy vấn	Cập nhật điểm	Bộ nhớ
Cây Chỉ Số Nhị Phân (Fenwick Tree)	$O(N)$	$O(\log N)$ (rất phức tạp cho Min/Max)	$O(\log N)$	$O(N)$
Bảng Thừa (Sparse Table)	$O(N \log N)$	$O(1)$	Không hỗ trợ	$O(N \log N)$
Bảng Thừa Rời rạc (Disjoint Sparse Table)	$O(N \log N)$	$O(1)$ (cho cả phép Cộng)	Không hỗ trợ	$O(N \log N)$

i Ghi chú:

****Nhận định thực chiến:****
 Khi một bài toán ****không có thao tác cập nhật (Static Data)**** và yêu cầu trả lời hàng triệu truy vấn đoạn với các hàm như `min`, `max`, `gcd`, `AND`, `OR`, ****Sparse Table** là cấu trúc dữ liệu tối ưu nhất hiện nay** nhờ thời gian truy vấn $O(1)$ tuyệt đối với hằng số thực thi cực nhỏ (chỉ 1 phép tính `log2` và 1 phép `min`).

1.2. Nguyên lý Lũy Thừa 2 (Binary Jumping Principle)

Mọi số nguyên dương $L \leq N$ đều có thể biểu diễn duy nhất dưới dạng tổng của các lũy thừa của 2:

$$L = 2^{k_1} + 2^{k_2} + \dots + 2^{k_m} \quad (k_1 > k_2 > \dots > k_m \geq 0)$$

Thay vì tiền xử lý kết quả cho mọi đoạn con khả dĩ ($O(N^2)$ đoạn), Bảng Thừa chỉ tiền xử lý các đoạn con có **độ dài đúng bằng lũy thừa của 2**:

$$\text{Độ dài} \in \{2^0, 2^1, 2^2, 2^3, \dots, 2^K\} \quad \text{với } K = \lfloor \log_2 N \rfloor$$

Định nghĩa Bảng Trạng Thái:

Gọi `st[k][i]` là kết quả truy vấn trên đoạn con bắt đầu tại chỉ số *i* và có độ dài chính xác là 2^k :

$$st[k][i] = \text{Query}(i, i + 2^k - 1)$$

- Phạm vi quản lý của `st[k][i]` là nửa mở $[i, i + 2^k)$.
- Với mỗi phần tử *i*, ta lưu trữ tối đa $K + 1 \approx \log_2 N + 1$ khoảng.
- Tổng số phần tử cần lưu trữ trong bảng: $N \cdot (\lfloor \log_2 N \rfloor + 1) \approx O(N \log N)$.

Chỉ số:	0	1	2	3	4	5	6	7	
	[---]								<code>st[0][0]</code> : độ dài $2^0 = 1$
	[-----]								<code>st[1][0]</code> : độ dài $2^1 = 2$
	[-----]								<code>st[2][0]</code> : độ dài $2^2 = 4$
	[-----]								<code>st[3][0]</code> : độ dài $2^3 = 8$

2. Quy Hoạch Động Xây Dựng Bảng Thừa $O(N \log N)$

Một đoạn con độ dài 2^k luôn có thể chia đôi một cách hoàn hảo thành **hai đoạn con rời nhau có độ dài bằng nhau là 2^{k-1}** :

$$[i, i + 2^k - 1] = [i, i + 2^{k-1} - 1] \cup [i + 2^{k-1}, i + 2^k - 1]$$

Đoạn lớn độ dài 2^k : `[----- st[k][i] -----]`
Đoạn con trái (2^{k-1}): `[----- st[k-1][i] -----]`
Đoạn con phải (2^{k-1}): `[-- st[k-1][i + 2^{k-1}] --]`

2.1. Công thức Truy Hồi Chuyển Trạng Thái

- Cơ sở quy hoạch động ($k = 0$):

$$\text{st}[0][i] = A[i] \quad \forall i \in [0, N - 1]$$

- Bước chuyển trạng thái ($k \geq 1$):

$$\text{st}[k][i] = \min(\text{st}[k-1][i], \text{st}[k-1][i + 2^{k-1}])$$

(Điều kiện: $i + 2^k - 1 < N$, các vị trí vượt quá N không cần tính).

2.2. Thứ Tự Duyệt Vòng Lặp & Tối Ưu Cache Locality

🚩 Quan trọng:

****Tối ưu hóa Cache CPU:****

Trong C++, mảng đa chiều được lưu trữ liên tục theo thứ tự hàng (Row-major order).

Để các truy cập bộ nhớ liên tục trong vòng lặp trong cùng, mảng Bảng Thừa phải được khai báo theo cấu trúc:

$$\text{st}[k][i] \quad \text{thay vì} \quad \text{st}[i][k]$$

Khi duyệt k ở vòng ngoài và i ở vòng trong, CPU sẽ nạp tuần tự các phần tử `st[k][i]`, `st[k][i+1]`, ... vào Cache Line L1/L2, giảm thiểu tối đa Cache Misses và tăng tốc độ build lên ****gấp 2.5 lần****!

```
// Thứ tự lặp chuẩn mực O(N log N)
for (int k = 1; (1 << k) <= n; ++k) {
    int len = 1 << (k - 1);
    for (int i = 0; i + (1 << k) <= n; ++i) {
        st[k][i] = std::min(st[k-1][i], st[k-1][i + len]);
    }
}
```

3. Cơ Chế Truy Vấn $O(1)$: Phép Toán Lũy Đẳng (Idempotent Operations)

3.1. Tính Chất Lũy Đẳng là gì?

Một phép toán hai ngôi $\circ$ được gọi là **lũy đẳng (Idempotent)** nếu việc áp dụng phép toán đó lên cùng một toán tử nhiều lần không làm thay đổi giá trị:

$$x \circ x = x \quad \forall x$$

Các phép toán lũy đẳng kinh điển trong Lập trình thi đấu:

1. Phép tìm phần tử nhỏ nhất: $\min(x, x) = x$.
2. Phép tìm phần tử lớn nhất: $\max(x, x) = x$.
3. Phép tìm ước chung lớn nhất: $\gcd(x, x) = x$.
4. Phép toán bit OR: $x \mid x = x$.
5. Phép toán bit AND: $x \& x = x$.

3.2. Kỹ Thuật Hai Đoạn Phủ Chồng (Overlapping Intervals)

Xét truy vấn đoạn $[L, R]$ với độ dài $\text{len} = R - L + 1$.

Gọi k là số mũ nguyên lớn nhất sao cho lũy thừa $2^k \leq \text{len}$:

$$k = \lfloor \log_2(R - L + 1) \rfloor$$

Khi đó, đoạn $[L, R]$ có thể được phủ kín hoàn toàn bởi **hai đoạn con có độ dài 2^k** :

1. Đoạn con xuất phát từ L : $[L, L + 2^k - 1] = \text{st}[k][L]$.
2. Đoạn con kết thúc tại R : $[R - 2^k + 1, R] = \text{st}[k][R - 2^k + 1]$.

```
Truy vấn [L, R]:           [=====]
Đoạn trái st[k][L]:        [-----]
Đoạn phải st[k][R - 2^k + 1]: [-----]
Phần giao nhau (Overlapping): [----] (Không ảnh hưởng đến Min!)
```

Do tính lũy đẳng:

$$\min_{i=L}^R A[i] = \min(\text{st}[k][L], \text{st}[k][R - 2^k + 1])$$

Vì cả hai đoạn con đều đã được tính sẵn trong bảng `st`, phép truy vấn chỉ gồm:

1. Tính $k = \lfloor \log_2(\text{len}) \rfloor$ trong $O(1)$.
2. Lấy giá trị 2 ô và gọi hàm `min` $\implies$ **Độ phức tạp chính xác $O(1)$!**

3.3. Tính $\lfloor \log_2(X) \rfloor$ trong $O(1)$ Siêu Tốc

Trong CP, việc gọi hàm toán học số thực `std::log2(len)` sẽ gây chậm do chuyển đổi dấu phẩy động và có thể sai số làm tròn. Có 3 cách tiếp cận chuẩn mực:

1. Mảng tiền xử lý (Lookup Table):

```
cpp std::vector<int> lg(n + 1); lg[1] = 0; for (int i = 2; i <= n; ++i) lg[i] = lg[i / 2] + 1;
```

2. Hàm nội tại GCC `__builtin_clz` (Count Leading Zeros):

Số mũ $k = 31 - \text{__builtin_clz}(\text{len})$. Đây là lệnh đơn kỳ của CPU (BSR trên x86/x64), thực thi trong đúng 1 chu kỳ máy ($O(1)$ tuyệt đối).

3. Chuẩn C++20 `std::__lg` hoặc `<bit>`:

```
cpp #include <bit> int k = std::bit_width(static_cast<unsigned>(len)) - 1; // hoặc nội tại GCC:
int k = std::__lg(len);
```

4. Bảng Trace Step-by-Step Trực Quan Hóa

Xét mảng A gồm $N = 10$ phần tử:

$$A = [4, 7, 2, 8, 1, 9, 3, 5, 6, 2]$$

4.1. Bảng Trạng Thái `st[k][i]`

k	Độ dài 2^k	$i = 0$	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$	$i = 7$	$i = 8$	$i = 9$
0	1	4	7	2	8	1	9	3	5	6	2
1	2	4	2	2	1	1	3	3	5	2	—
2	4	2	1	1	1	1	3	2	—	—	—
3	8	1	1	1	—	—	—	—	—	—	—

Giải thích một số ô:

- $st[1][1] = \min(st[0][1], st[0][2]) = \min(7, 2) = 2$.
- $st[2][0] = \min(st[1][0], st[1][2]) = \min(4, 2) = 2$ (đoạn $[0 \dots 3]$ gồm 4, 7, 2, 8).
- $st[3][0] = \min(st[2][0], st[2][4]) = \min(2, 1) = 1$ (đoạn $[0 \dots 7]$).

4.2. Trace Truy Vấn Đoạn $L = 2, R = 7$ (Đoạn $[2, 8, 1, 9, 3, 5]$)

1. Độ dài: $len = 7 - 2 + 1 = 6$.
2. Số mũ: $k = \lfloor \log_2(6) \rfloor = 2$ (vì $2^2 = 4 \leq 6 < 2^3 = 8$).
3. Hai đoạn con độ dài $2^2 = 4$:
4. Đoạn 1: Bắt đầu từ $L = 2 \implies st[2][2]$ (quản lý chỉ số 2, 3, 4, 5 với các giá trị 2, 8, 1, 9 $\implies \min = 1$).
5. Đoạn 2: Kết thúc tại $R = 7 \implies$ Bắt đầu từ $7 - 4 + 1 = 4 \implies st[2][4]$ (quản lý chỉ số 4, 5, 6, 7 với các giá trị 1, 9, 3, 5 $\implies \min = 1$).
6. Kết quả: $\min(st[2][2], st[2][4]) = \min(1, 1) = 1$.

Chính xác và chỉ tốn 2 phép đọc mảng!

5. Truy Vấn Phép Toán Kết Hợp Bất Kỳ: Phân Tích Nhị Phân $O(\log N)$

Nếu phép toán **không có tính lũy đẳng** (ví dụ: Tính tổng đoạn $\sum_{i=L}^R A[i]$, Phép nhân ma trận, Phép nhân modulo M):

$$x + x \neq x$$

Ta không thể để hai đoạn con phủ chồng lên nhau vì các phần tử ở phần giao sẽ bị cộng hai lần!

Giải pháp Phân Tích Nhị Phân

Ta phân rã độ dài $\text{len} = R - L + 1$ thành tổng các lũy thừa của 2 không giao nhau:

```
long long query_sum(int L, int R) {
    long long sum = 0;
    int cur = L;
    int len = R - L + 1;
    for (int k = 0; (1 << k) <= len; ++k) {
        if ((len >> k) & 1) {
            sum += st[k][cur];
            cur += (1 << k);
        }
    }
    return sum;
}
```

- Độ phức tạp: $O(\log N)$ phép tính.
- Mặc dù cách này chạy được, nhưng nếu cần truy vấn tổng tĩnh $O(1)$, ta dùng **Mảng Tiền Tố (Prefix Sum)** ($O(1)$ thời gian, $O(N)$ bộ nhớ).
- Tuy nhiên, nếu phép toán **không có phần tử nghịch đảo** (ví dụ: Nhân ma trận non-invertible, Ghép khâu) và vẫn muốn truy vấn $O(1)$, ta sử dụng cấu trúc **Disjoint Sparse Table** ở Phần 7!

6. Sparse Table 2D (RMQ Trên Lưới Ma Trận $N \times M$)

Trong các bài toán hình học lưới hoặc ma trận 2D tĩnh, ta cần tìm **min** / **max** trong hình chữ nhật con $[x_1, y_1] \dots [x_2, y_2]$ trong $O(1)$.

6.1. Cấu Trúc Bảng 4 Chiều

Gọi `st[kx][ky][i][j]` là giá trị cực trị trong hình chữ nhật có góc trái trên tại (i, j) và kích thước $2^{kx} \times 2^{ky}$.

- Kích thước bảng: $(\log_2 N + 1) \times (\log_2 M + 1) \times N \times M$.

- Thời gian tiền xử lý: $O(NM \log N \log M)$.

6.2. Quy Hoạch Động Xây Dựng Bảng 2D

1. Khởi tạo $kx = 0, ky = 0$: `st[0][0][i][j] = A[i][j]`.

2. Xây dựng theo chiều y khi $kx = 0$:

$$st[0][ky][i][j] = \min(st[0][ky-1][i][j], st[0][ky-1][i][j+2^{ky-1}])$$

3. Xây dựng theo chiều x cho mọi $kx \geq 1$:

$$st[kx][ky][i][j] = \min(st[kx-1][ky][i][j], st[kx-1][ky][i+2^{kx-1}][j])$$

6.3. Truy Vấn Hình Chữ Nhật Trong $O(1)$

Với truy vấn hình chữ nhật $[x_1, y_1] \dots [x_2, y_2]$:

- Đặt $kx = \lfloor \log_2(x_2 - x_1 + 1) \rfloor$ và $ky = \lfloor \log_2(y_2 - y_1 + 1) \rfloor$.

- Ta phủ hình chữ nhật cần truy vấn bằng **4 hình chữ nhật con kích thước $2^{kx} \times 2^{ky}$** tại 4 góc:

1. Góc trái trên: $st[kx][ky][x_1][y_1]$

2. Góc phải trên: $st[kx][ky][x_1][y_2 - 2^{ky} + 1]$

3. Góc trái dưới: $st[kx][ky][x_2 - 2^{kx} + 1][y_1]$

4. Góc phải dưới: $st[kx][ky][x_2 - 2^{kx} + 1][y_2 - 2^{ky} + 1]$

- Lấy min của 4 giá trị này cho kết quả chính xác trong $O(1)$!

7. Bảng Thưa Rời Rạc (Disjoint Sparse Table - DST)

💡 Mẹo / Tối ưu:

****Định cao Cấu trúc dữ liệu Tĩnh:****

Sparse Table thông thường chỉ truy vấn được $O(1)$ cho phép toán lũy đẳng.

****Disjoint Sparse Table (DST)**** là một biến thể cấu trúc dạng Cây Nhị Phân cho phép truy vấn **$O(1)$** cho **BẤT KỲ PHÉP TOÁN KẾT HỢP NÀO** (Monoid: Tổng, Phép nhân ma trận, Ghép xâu, Modulo không có nghịch đảo)!

Tầng 3: [----- Gốc -----] (Trung tâm mid)
Tầng 2: [----- Con trái -----][----- Con phải -----]
Tầng 1: [--- C1 ---][--- C2 ---][--- C3 ---][--- C4 ---]

7.1. Nguyên Lý Hoạt Động

- Xét mảng được đếm kích thước lên lũy thừa của 2: $2^K \geq N$.
- Tại mỗi tầng k (tương ứng với các khối kích thước 2^{k+1}):
- Khối $[l, r]$ có tâm chia đôi tại $mid = (l + r) / 2$.
- Ta tính **mảng hậu tố (Suffix)** từ mid lùi về l : $table[k][i] = A[i] \circ \dots \circ A[mid]$ ($i \leq mid$).
- Ta tính **mảng tiền tố (Prefix)** từ $mid + 1$ tiến về r : $table[k][j] = A[mid + 1] \circ \dots \circ A[j]$ ($j > mid$).

7.2. Truy Vấn $O(1)$ Thần Sâu

Với mọi đoạn truy vấn $[L, R]$ ($L < R$):

- Vị trí bit khác nhau cao nhất giữa L và R :

$$k = \text{MSB}(L \oplus R) = \lfloor \log_2(L \oplus R) \rfloor$$

- Nhờ cấu trúc nhị phân, tầng k chính là tầng mà L và R nằm về hai phía khác nhau của điểm phân cách mid !

- Do đó:

$$\text{Query}(L, R) = \text{table}[k][L] \circ \text{table}[k][R]$$

(Đoạn $[L, mid]$ lấy từ mảng hậu tố, đoạn $[mid + 1, R]$ lấy từ mảng tiền tố).

- Chỉ đúng **1 phép XOR, 1 phép lấy MSB và 1 phép tính toán hai ngôi**, đạt tốc độ $O(1)$ nhanh nhất trong các CTDL nâng cao!

8. Các Mô Hình Ứng Dụng Thực Chiến Trong CP

Mô hình 1: Tìm Tổ Tiên Chung Gần Nhất (LCA) Trong $O(1)$

Bằng cách trải phẳng cây qua Full Euler Tour (độ dài $2N - 1$ bước duyệt đỉnh), bài toán tìm $\text{LCA}(u, v)$ tương đương với tìm đỉnh có độ sâu nhỏ nhất (RMQ) trên dãy Euler Tour giữa lần xuất hiện đầu tiên của u và v :

$$\text{LCA}(u, v) = \text{EulerTour} \left[\arg \min_{i=\min(\text{pos}[u], \text{pos}[v])}^{\max(\text{pos}[u], \text{pos}[v])} \text{depth}[\text{EulerTour}[i]] \right]$$

- Sử dụng Sparse Table lưu chỉ số hoặc lưu cặp (depth, node):
- Tiền xử lý: $O(N \log N)$ (hoặc $O(N)$ bằng kỹ thuật Farach-Colton & Bender).
- Trả lời truy vấn LCA trong $O(1)$!

Mô hình 2: Đếm Số Đoạn Con Có $\text{gcd} = x$

Hàm $\text{gcd}(L, R)$ là một hàm giảm dần khi mở rộng R . Với một L cố định, giá trị $\text{gcd}(L, R)$ khi tăng R chỉ có thể giảm tối đa $\log_2(\max A_i)$ lần (mỗi lần giảm ít nhất chia cho 2).

- Bằng cách dùng Sparse Table truy vấn $\text{gcd}(L, R)$ trong $O(1)$ kết hợp Tìm kiếm Nhị phân, ta có thể tìm nhanh điểm kết thúc của mỗi đoạn có cùng giá trị gcd .
- Duyệt qua toàn bộ mảng chỉ mất $O(N \log N \log(\max A))$ thay vì $O(N^2)$.

Mô hình 3: Hai Bảng Thừa Song Song (Min & Max)

Nhiều bài toán yêu cầu tìm số cặp (L, R) sao cho $\max(A[L \dots R]) = \min(B[L \dots R])$.

- Vì $\max(A)$ là hàm không giảm theo R , còn $\min(B)$ là hàm không tăng theo R , hiệu $f(R) = \max(A[L \dots R]) - \min(B[L \dots R])$ là một hàm đơn điệu tăng theo R .

- Dựng 2 Sparse Table cho A (Max) và B (Min). Với mỗi L , dùng `std::lower_bound` / binary search trên $f(R)$ để đếm số lượng vị trí R thỏa mãn $f(R) = 0$ trong $O(\log N)$ mỗi $L \implies$ Tổng thời gian $O(N \log N)$.

9. Cạm Bẫy Phòng Thi & Kỹ Thuật Tối Ưu (Traps & Tips)

1. Trần Màng Số Mũ K :

Với $N = 2 \times 10^5$, $\log_2 N \approx 17.6 \implies$ Kích thước chiều thứ nhất phải tối thiểu là 18 hoặc 19 (`st[20][N]`). Đặt thiếu kích thước sẽ gây Runtime Error (SIGSEGV).

2. Kích thước Cận Phải Trong Vòng Lặp Xây Dựng:

Điều kiện dừng ở vòng lặp trong phải là `i + (1 << k) <= n` (với 0-indexed), tránh để chỉ số vượt quá phạm vi mảng.

3. Sparse Table Lưu Vị Trí (Index RMQ):

Khi đề bài yêu cầu tìm vị trí của phần tử nhỏ nhất đầu tiên / cuối cùng (First/Last Minimum Index), không được lưu giá trị trần, mà phải lưu chỉ số *idx*:

```
cpp auto combine = [&](int i, int j) { return (a[i] < a[j] || (a[i] == a[j] && i < j)) ? i : j; };
```

4. Không Dùng Sparse Table Cho Bài Toán Có Update:

Nếu bài toán có thao tác cập nhật giá trị phần tử (`A[i] = x`), một cập nhật có thể ảnh hưởng tới $O(N)$ ô trong Sparse Table $\implies$ Bắt buộc phải chuyển sang dùng Segment Tree hoặc Fenwick Tree.

10. Tài liệu Tham khảo & Đọc thêm

1. CP-Algorithms:

2. [Sparse Table](#) — Phân tích RMQ $O(1)$ và phép toán lũy đẳng (Idempotent).

3. [VNOI Wiki](#):

4. [Bảng thưa & RMQ](#) — Cài đặt bảng thưa 1D, 2D và Disjoint Sparse Table.

5. [Codeforces Community](#):

6. [Disjoint Sparse Table Tutorial](#) — Truy vấn $O(1)$ cho mọi Monoid phép toán kết hợp.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agent AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.